

Estruturas de Repetição

O Poder da Automação com Laços

Chega de código repetido: automatizando tarefas com while, do...while, for clássico e o controle de fluxo com break e continue.

Docente: Jeronimo Silva · Turma 1082 · Senac Centro (Tarde · 2026)

O Que Faremos Hoje

1. Por Que Usar Laços?

O princípio DRY (Don't Repeat Yourself), o conceito de iteração e o perigo real do loop infinito.

2. O Laço while (Enquanto)

Repetições baseadas em condições dinâmicas: validação de senhas e menus que repetem até o usuário sair.

3. O Laço do...while

Garantindo a execução de pelo menos uma vez antes de testar a condição de continuidade.

4. O Laço for Clássico

Contadores estruturados (início, condição e incremento) para tabuadas, contagens e limites fixos.

Integração com Codespaces: Exemplos com contadores e entrada interativa com `@inquirer/prompts` .

Termos e Palavras-Chave da Aula

Palavra-Chave	O que significa na prática?
Iteração	Cada volta ou ciclo completo de execução dentro de um laço de repetição.
Condição de Parada	A regra booleana que, ao se tornar <code>false</code> , encerra o laço com segurança.
Loop Infinito	Bug em que a condição nunca vira falsa, fazendo o programa travar consumindo 100% da CPU.
<code>while</code> (Enquanto)	Testa a condição no início . Se for falsa logo de primeira, nem entra no bloco.
<code>do...while</code>	Executa o bloco primeiro e só testa a condição no final . Roda pelo menos 1 vez.
<code>for</code>	Agrupa em uma única linha: inicialização da variável, condição de parada e incremento.
<code>break</code>	Interrompe bruscamente e sai de vez do laço, sem rodar as voltas restantes.
<code>continue</code>	Pula o restante da volta atual e salta direto para a próxima iteração do laço.

O Problema do Código Repetido

Imagine que você precisa imprimir uma contagem de 1 até 5 no terminal:

❌ Sem Laço (Código Engessado)

```
console.log("Número: 1");  
console.log("Número: 2");  
console.log("Número: 3");  
console.log("Número: 4");  
console.log("Número: 5");  
// E se o cliente pedir até 10.000?
```

✅ Com Laço (Automatizado)

```
let numero = 1;  
  
while (numero ≤ 5) {  
  console.log(`Número: ${numero}`);  
  numero++; // Passo obrigatório!  
}
```



Princípio DRY (Don't Repeat Yourself): Computadores existem para fazer tarefas repetitivas com velocidade máxima e sem cansar.

BLOCO 1 · O LAÇO WHILE

O Laço `while` (Enquanto For Verdade)

O comando `while` repete o bloco de código `{ ... }` enquanto a sua condição for verdadeira:

```
let contador = 1; // 1. Variável de Controle (Início)

while (contador ≤ 3) { // 2. Condição de Parada (Teste)
  console.log(`Volta atual do loop: ${contador}`);
  contador++; // 3. Modificação da Variável (Passo)
}

console.log("🏁 Loop finalizado com sucesso!");
```

```
Volta 1: contador vale 1 (1 ≤ 3 é true) → Imprime e soma → contador vira 2
Volta 2: contador vale 2 (2 ≤ 3 é true) → Imprime e soma → contador vira 3
Volta 3: contador vale 3 (3 ≤ 3 é true) → Imprime e soma → contador vira 4
Volta 4: contador vale 4 (4 ≤ 3 é false) → SAI DO LAÇO!
```

Cuidado: O Fantasma do Loop Infinito!

O que acontece se você esquecer de atualizar o contador (`contador++`)?

```
let contador = 1;

while (contador ≤ 3) {
  console.log(`Contador: ${contador}`);
  // ESQUECEU 0: contador++;
}
```

💣 **O Que Acontece?** O valor de `contador` será eternamente `1`. A condição `1 ≤ 3` será eternamente `true`. O terminal vai travar, a ventoinha da CPU vai acelerar e o Node.js não vai parar até você pressionar **Ctrl + C**!

Lembre-se: Todo laço DEVE ter um caminho garantido para que sua condição se torne falsa.

Validando Entradas do Usuário

O `while` brilha quando não sabemos quantas vezes o usuário vai errar antes de acertar:

```
import { input } from '@inquirer/prompts';

let senha = "";

// Enquanto a senha digitada for diferente do segredo "senac123":
while (senha !== "senac123") {
  senha = await input({ message: "Digite a senha de acesso ao cofre:" });

  if (senha !== "senac123") {
    console.log("✗ Senha incorreta! Tente novamente.");
  }
}

console.log("🔓 Acesso concedido! Cofre liberado.");
```

O laço pode rodar 1 vez (se ele acertar de primeira) ou 50 vezes (se ele continuar errando).

O Laço `do...while` (Faça Primeiro, Teste Depois)

A diferença essencial para o `while` tradicional: o código roda **pelo menos uma vez**, mesmo que a condição seja falsa desde o início!

`while` (Teste no Início)

```
let numero = 10;

while (numero < 5) {
  // 10 < 5 é FALSE logo de cara!
  console.log("Nunca vai rodar!");
}
```

`do...while` (Teste no Final)

```
let numero = 10;

do {
  // Executa antes de perguntar!
  console.log("Roda pelo menos 1 vez!");
} while (numero < 5);
```

 **Caso de Uso Perfeito:** Menus interativos onde você precisa exibir as opções na tela antes de saber a resposta do usuário.

BLOCO 3 · O LAÇO FOR CLÁSSICO

O Laço `for` : A Anatomia do Contador

Quando sabemos exatamente o ponto de partida e o limite final, o `for` consolida tudo em uma única linha limpa:

```
// for (INICIALIZAÇÃO ; CONDIÇÃO DE PARADA ; INCREMENTO / PASSO)
for (let i = 1; i ≤ 5; i++) {
  console.log(`Contando: ${i}`);
}
```

1. `let i = 1;`

Inicialização: Executa *apenas uma vez* no instante em que o laço é acionado.

2. `i ≤ 5;`

Condição: Verificada antes de cada volta. Se for `false`, o laço encerra.

3. `i++`

Passo: Executado automaticamente ao término de cada volta (soma +1 ao contador).

Curiosidade: Por convenção mundial na programação, a variável de contagem de loops é chamada de `i` (de índice / index).

Construindo a Tabuada do 7 com `for`

Veja como uma operação matemática inteira é gerada em apenas 4 linhas de código:

```
const multiplicador = 7;

console.log(`≡ TABUADA DO ${multiplicador} ≡`);

for (let i = 1; i ≤ 10; i++) {
  const resultado = multiplicador * i;
  console.log(`${multiplicador} x ${i} = ${resultado}`);
}

console.log("=====");
```

```
≡ TABUADA DO 7 ≡
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
=====
```

Interrupção Imediata com `break`

O comando `break` funciona como um "freio de emergência": ele encerra o laço imediatamente, sem executar mais nenhuma iteração:

```
// Procurando um número da sorte entre 1 e 20:  
for (let i = 1; i ≤ 20; i++) {  
  if (i === 7) {  
    console.log(`🎯 Encontrei o número ${i}! Encerrando a busca.`);  
    break; // Sai do laço agora mesmo!  
  }  
  
  console.log(`Verificando item ${i}...`);  
}  
  
console.log("Processamento concluído.");
```

Ao encontrar o 7, o loop para na hora. As voltas de 8 até 20 nem chegam a ser processadas!

Saltando Iterações com `continue`

Diferente do `break` (que destrói o laço), o `continue` apenas cancela o restante da volta atual e pula direto para a próxima:

```
// Imprimindo apenas números ímpares:
for (let i = 1; i ≤ 6; i++) {
  if (i % 2 === 0) {
    // Se o número for PAR, pula a linha de baixo!
    continue;
  }

  console.log(`Número ímpar encontrado: ${i}`);
}
```

Saída no terminal:

```
Número ímpar encontrado: 1
Número ímpar encontrado: 3
Número ímpar encontrado: 5
```

Qual Laço Devo Escolher?

Use esta bússola rápida para saber qual estrutura aplicar em cada projeto:

Estrutura	Quando devo utilizar?	Exemplo Real
<code>for</code>	Quando você sabe com antecedência quantas voltas serão necessárias.	Tabuadas, contagens de 1 a 100, leitura de listas com tamanho fixo.
<code>while</code>	Quando o laço depende de uma condição externa que pode demorar a mudar.	Esperar o usuário digitar a senha certa, conexão com rede, jogo até perder.
<code>do...while</code>	Quando o bloco precisa rodar ao menos uma vez antes de qualquer checagem.	Menus que mostram opções na tela antes de ler a resposta do terminal.

Regra Prática: Se envolve um contador simples com início e fim, use `for`. Se envolve esperar algo acontecer, use `while`.

Desafio: Tabuada Personalizada Interativa

Crie o arquivo `tabuada.js` com o seguinte comportamento:

1. Peça ao usuário qual número ele deseja consultar via `number()` do `@inquirer/prompts`.
2. Peça até qual limite a tabuada deve ir (ex: até 10, até 20).
3. Use um laço `for` para calcular e imprimir a tabuada completa formatada.

Exemplo de Execução:

```
? Qual tabuada deseja consultar? 9
? Até qual número multiplicar? 5
```

```
=== TABUADA DO 9 (Até 5) ===
```

```
9 x 1 = 9
```

```
9 x 2 = 18
```

```
9 x 3 = 27
```

```
9 x 4 = 36
```

```
9 x 5 = 45
```

```
=====
```

Desafio Final: O Caixa do Supermercado (PDV)

Crie o arquivo `caixa_pdv.js` simulando a passagem de compras em um supermercado:

1. Inicie as variáveis: `let total = 0;` e `let continuar = true;` .
2. Use um laço `while (continuar)` que:
 - Pergunta o valor do produto via `number()` .
 - Soma o valor ao `total` .
 - Pergunta via `confirm()` se há mais produtos a passar.
 - Se o usuário responder que não, define `continuar = false;` .
3. Ao sair do laço, imprima o valor total acumulado e a quantidade de itens.

Desafio Extra: Se o valor digitado for menor ou igual a 0, exiba `"Valor inválido!"` e use `continue` para não somar ao total.

SÍNTESE & ENCERRAMENTO

O Que Dominamos na Aula 04?

1. Laços Dinâmicos

Controle de fluxo repetitivo com `while` e `do...while` baseado em condições de parada seguras.

2. Contadores com for

Estruturação de repetições com início, condição e incremento para tabuadas e contagens precisas.

3. Controle de Fluxo

Interrupções cirúrgicas com `break` e saltos de iteração com `continue`.

PRÓXIMA AULA · AULA 05

Coleções de Dados em JavaScript

O Poder dos Arrays e Objetos

Como guardar listas de clientes, estoques de produtos e carrinhos inteiros em uma única variável.

Parabéns pelo avanço e até a próxima aula! 🚀